



NOVEMBER 15, 2025

WSU_FORUM PROJECT REPORT

CSC 4110 ; GROUP 19

Team Members:

Alghada Badani

Eman Jibril

Sanjeda Khan

Jean Dilloway

Ariq Chowdhury

Product Manager: Jason Myer

1. Introduction

1.1 Project summary

WSU Forum is a mock web application designed to improve communication, collaboration, and community engagement across Wayne State University. The platform allows students and faculty to share posts, participate in discussions, follow topics, and stay informed about campus events.

This project demonstrates core software engineering concepts like:

- Requirements gathering
- UI design
- Persistent data storage
- Documentation
- Design principles

1.2 Purpose & objective

The purpose of this project is to build a simple, easy-to-use communication hub for the WSU community.

The main objectives of this project are:

- Create a clean front-end interface based on our mockups
- Allow users to create posts and store them so they don't disappear after refreshing
- Follow basic software design ideas like encapsulation, and SOLID
- Put together documentation (user guide, diagrams, requirements, etc.)
- Practice a real development workflow using Scrum

1.3 Value to Stakeholder (ROI)

WSU Forum gives value by making communication easier for students, faculty, and anyone involved on campus. Instead of information being scattered between emails, group chats, and different platforms, everything can be viewed in one place.

For students, it helps them stay updated on events, posts, and topics they care about.

For faculty, it's an easy way to share announcements or answer questions.

From a project point of view, the "ROI" comes from creating a basic tool that solves a real communication gap

2. Stakeholder Review

2.1 Identified Stakeholders

For this project, the main stakeholders are the people who would actually use or benefit from WSU Forum.

These include:

- WSU Students : want a place to share posts, find info, and interact
- Faculty Members : want an easy way to share announcements or updates
- IT / Admin Staff : care about system reliability and organization
- Campus Organizations : can use the platform to promote events or discussions

2.2 Stakeholder Needs

After looking at what the different groups might need, we focused on the following:

- A simple and organized way to post announcements or updates
- A feed where users can quickly see what's happening
- A topics section to follow interests or categories
- A way to save posts so they don't disappear after refreshing
- Easy navigation so new users don't get confused

2.3 How the Application Meets These Needs

WSU Forum was built to match the needs we identified.

Here's how:

- The Feed page lets users easily scroll through recent posts and updates
- The Topics page helps organize content by category
- The Create Post form gives users a way to share new announcements or discussions
- We added persistent storage, so posts stay saved
- The UI layout is clean and simple, making the platform easy to learn and use

Overall, the application solves the basic communication needs of students and faculty while showing how a more complete campus communication tool could work.

3. User Guide

3.1 Licensing Statement

This project was created by Group 19 for the CSC 4110 course at Wayne State University. It is for class use only and isn't meant for real deployment, distribution, or commercial use.

3.2 Indemnification Statement

Because this is a mock application, the developers are not responsible for any data loss or problems that come from using the project.

It is only meant for demonstration and learning purposes.

3.3 End-User Instructions

Once the application is deployed through Vercel, users will be able to access it directly through a shared link.

The basic steps for using the app are:

1. Open the website using the provided Vercel link
2. Use the navigation menu to move between pages such as Feed, Topics, and Create Post
3. To add a post, go to the Create Post page, fill out the form, and submit
4. Your post will appear on the Feed page after submitting
5. Since the app uses persistent storage, posts should remain visible even after refreshing or reopening the page

4. Application Behavior & Storage

4.1 App Behavior (User Interactions)

WSU Forum is designed to be simple and easy to use. Here's how users interact with the app:

- Users can click Create Post to write and submit a new message or event.

- Once submitted, the post appears instantly on the Feed page.
- Users can switch between pages like Feed and Topics using the sidebar.
- The layout updates based on what page the user selects.

Overall, the app reacts to user actions in real time so everything feels responsive

4.2 Persistent Data Storage

To make sure posts don't disappear after refreshing the page, the app uses **persistent storage**. This means:

- Posts are saved in localStorage or a JSON-based data structure.
- When a user submits a post, it gets stored instead of just shown temporarily.
- When the page reloads, the app automatically loads all saved posts back onto the Feed.

This keeps the content consistent for the user even if they close or refresh the browser.

4.3 Data Flow Summary

Here's a simple breakdown of how data moves through the app:

1. User writes a post → clicks submit
2. The app validates the input
3. The Controller logic saves the post into localStorage
4. The Feed page (View) updates and displays the new post
5. When the page reloads, the app checks storage and loads all posts again

This structure follows the basic flow of the MVC pattern and keeps everything organized.

5. Front-End Implementation

5.1 Technologies Used (HTML, CSS, normalize.css, JS)

For the front end of WSU Forum, our team used the following technologies to build a clean and functional interface:

- React – used to structure the UI into components and manage page rendering

- Tailwind CSS – used for styling, layout, spacing, and creating a modern interface
- normalize.css – used to ensure consistent styling across different browsers
- Vercel – used for fast and automated deployment of the front-end application

5.2 UI Pages Implemented (Feed, Topics, Create Post)

WSU Forum includes several core pages to support user interactions:

- Home/Feed
- Topics
- Events
- Settings
- Account
- Inbox
- Notifications

5.3 Styling Approach (no inline styles, external CSS)

For styling, our team used:

- Tailwind CSS for consistent spacing, color, layout, and component styles
- External CSS files to keep styling separate from the HTML/JS logic

This approach keeps the UI clean, modular, and easy to update.

5.4 Responsiveness and Accessibility

WSU Forum was designed with basic responsiveness and accessibility in mind:

- **Responsiveness:**
 - We used Flexbox and Grid utilities from Tailwind CSS to make the layout adjust on different screen sizes
- **Accessibility:**
 - Pages maintain clear structure with readable headings and consistent spacing

- Buttons and input fields are labeled clearly to support easy navigation
- Colors and contrast are kept simple to ensure readability

6. Back-End / Application Logic

6.1 Data Structures

The backend organizes all core data using Django models. Each model represents an important part of the forum, such as:

- User
- Post
- Comment
- Like
- Subspace / Topic

These models help structure how information is stored and retrieved throughout the app.

6.2 Core Functions (*addPost, getPosts, etc.*)

The backend uses Django services and API views to handle main actions:

- `RegisterService().create()` : registers a new user
- `LoginService.post()` : processes login and returns authentication tokens
- `AllPostViews.post()` : creates a new post
- `AllPostViews.get()` : retrieves all posts for the homepage
- `SinglePostViews.get()` : returns detailed information about a selected post
- `ResetPassService.post()` : updates a user's password
- `SearchService.get()` : searches users, posts, and topics based on keywords

These functions form the main behavior of the backend.

6.3 Connection Between UI and Data

- The frontend sends requests to the backend through REST API endpoints.

- The backend responds by converting model data into JSON using serializers.
- The front end then updates the interface using the data returned by the backend.

This setup keeps communication clean and consistent.

6.4 Examples of App Logic Behavior

- When a user creates a post, the backend saves it, links it to the author, and sends the updated post list back to the frontend
- When a user clicks on a post, the backend returns full details, including comments, likes, and the author info
- When searching, the backend filters matching posts, users, and topics and returns results quickly through the search API.

7. Design Principles

7.1 Encapsulation

To keep the project organized, we separated the UI, data, and logic into different parts. Each part of the application has its own specific role, which makes the code easier to understand and maintain. This structure helps reduce errors and keeps different areas of the project from interfering with each other.

7.2 SOLID Principles

We used basic SOLID ideas to keep our project structured:

- **Single Responsibility:**
Each function or file has one main purpose (ex: adding a post, rendering the feed).
- **Open/Closed:**
The project can be expanded later (like adding comments or likes) without breaking the features that already work
- **Liskov Substitution:**
We used the same format for all posts, so anything that handles a post can also handle future post types

- **Interface Segregation:**

UI code is kept separate from data-handling code, so they don't depend on each other too much

- **Dependency Inversion:**

The UI (View) doesn't work directly with raw data, instead it goes through the Controller functions, which makes the code a lot cleaner

8. Scrum Summary

8.1 Overview of Scrum Process

Our team followed a simple Scrum process to stay organized throughout the project. Each sprint included task planning, short meetings to discuss progress, and updates to our engineering tools. Tasks were divided between front-end, back-end, and documentation roles. We used a shared project table, requirement tracker, and change log to keep track of everything. This helped us stay aligned and complete each part of the project on time.

8.2 Sprint/Meeting Logs

All Scrum and Sprint logs are maintained in our team's HTML blog-style page. They include meeting dates, action items, task assignments, blockers, and sprint updates.

Link to Scrum/Sprint Notes:

<https://alghada03.github.io/Scrum-Sprint-Blog/>

8.3 Team Progress Summary

During this sprint, our team made steady progress on both the UI and documentation.

Front-end developers completed mockups and began connecting form inputs to storage.

Back-end developers structured the core models and outlined the main API endpoints.

Documentation sections such as the User Guide, Stakeholder Review, and Application Behavior were completed. We also prepared updated slides for the final presentation.

Overall, Sprint 2 moved the project closer to full integration and final submission.

9. Appendix A – Requirements & Diagrams

All Inception, Elicitation, and Elaboration requirements are documented in the attached [Group 19 Updates.xlsx](#) file.

The requirement IDs in our [excel](#) file follow these naming rules:

- **i-#** → Inception requirements
- **e-#** → Elicitation requirements
- **eb-#** → Elaboration requirements
- **sp-#** → Specification requirements
- **n-#** → Negotiation requirements
- **R-#** → Technical task IDs used for the product backlog

9.1 Inception Requirements

The inception requirements include our initial understanding of the project, the problem we are trying to solve, and what the earliest version of WSU Forum needed to do. This includes identifying the problem statement, main goals, early assumptions, and who the project is being built for.

9.2 Elicitation Requirements

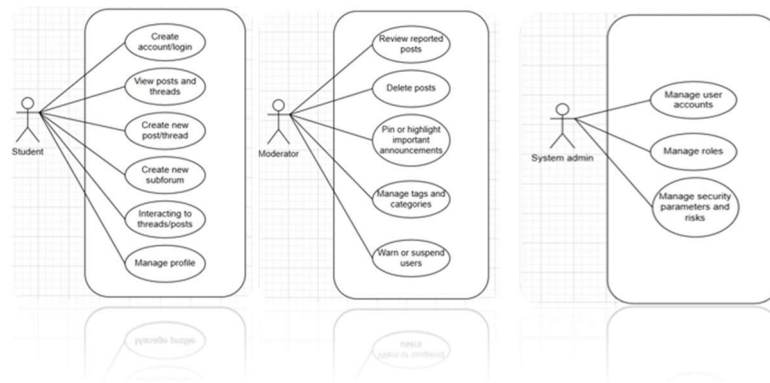
These requirements came from gathering more information, looking at diagrams, and refining our early ideas. They focus on what users expect from the system, what features need to exist, and what rules or constraints we must follow.

9.3 Elaboration Requirements

The elaboration requirements go deeper into how the system will actually work. They describe detailed behaviors, data needs, UI interactions, and features that must be included in the final version of WSU Forum. These requirements help define the final design before implementation.

9.4 Use Case Diagram

The Use Case Diagram shows the main interactions between users (students, faculty) and the system. It visually outlines what actions users can perform and how the system responds.

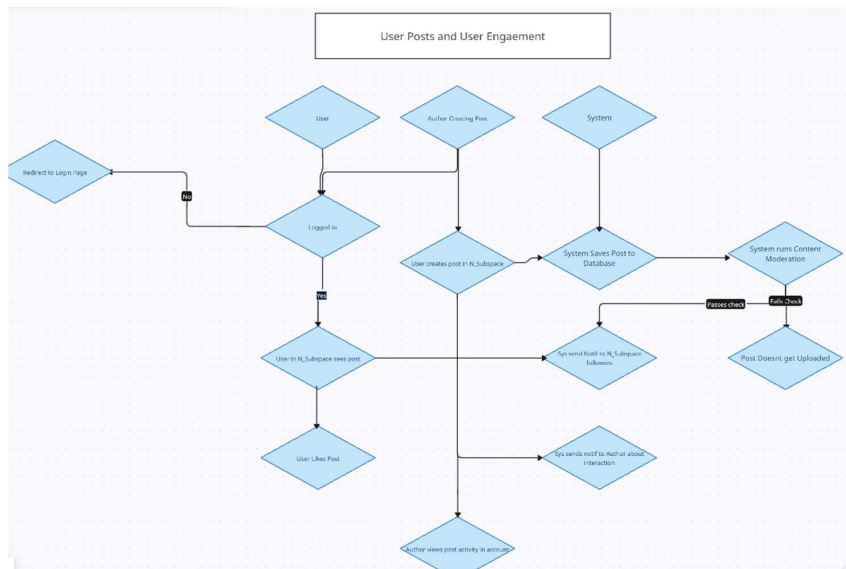


9.5 Combined Diagram (Sequence, Activity and Swim Lane)

We combined the Sequence Diagram, Activity Diagram, and Swim Lane Diagram into one unified workflow diagram. This combined diagram shows:

- The steps a user takes inside the system
- The order in which actions happen
- The roles/lanes responsible for each action (user, UI, logic, storage)
- The flow of data and behavior in the application

Using one combined diagram makes it easier to visualize the entire process without switching between separate diagrams.



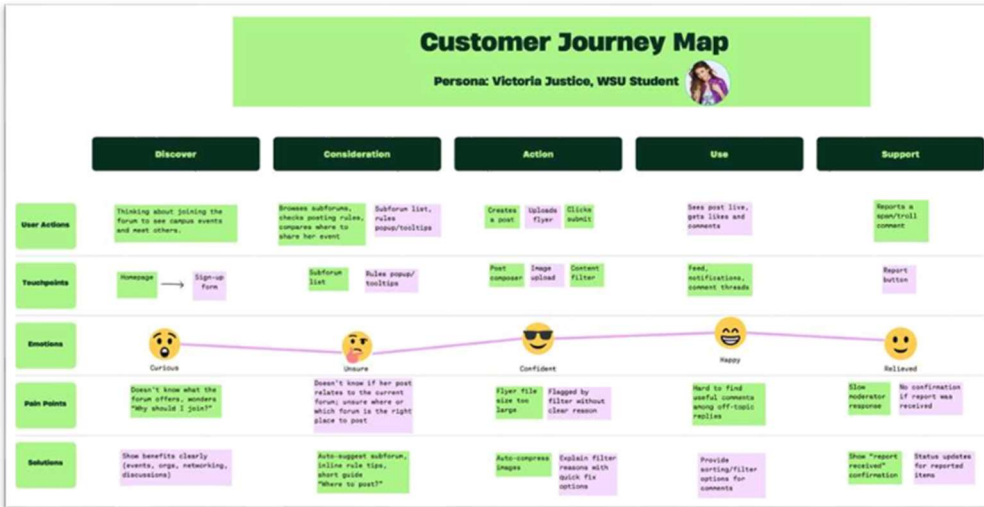
9.6 Persona

Our persona represents a typical WSU student member who would use the platform. This helps guide design decisions by focusing on real user needs.



9.7 Customer Journey Map

The journey map shows the experience a user has when using WSU Forum, from just discovering the app to creating posts and navigating topics. It highlights main points and goals.



9.8 Additional Engineering Tools/Forms

Additional tool(s) created:

- Scrum Notes [HTML](#) page